Aquí tienes el documento técnico redactado con instrucciones claras y directas, listo para que se lo envíes a tus compañeros de programación.

📄 Documentación Técnica: Sistema de Iluminación por Horarios (Easy to Fall)

Para: Equipo de Desarrollo (Programadores LÖVE 2D) **De:** Arte / Dev 2 **Objetivo:** Replicar exactamente la atmósfera visual y la estructura de capas de Procreate en el motor del juego, aplicando los filtros sobre el entorno y el jugador de forma dinámica según la hora del día.

1. Exportación de Archivos (Qué recibirán)

Los filtros no irán pegados al fondo. Para que la luz afecte al jugador cuando camina, exportaré cada capa de luz/sombra como un archivo .PNG transparente e independiente con resolución de 3840x1080 (o el tamaño de la pantalla).
Recibirán los archivos nombrados con este formato: [horario]_[numero_de_capa]_[modo_de_fusion].png. El número indica el orden de abajo hacia arriba.

2. El Orden de Dibujado (Z-Index)

En Procreate, las capas de abajo se dibujan primero y las de arriba al final. En LÖVE 2D deben seguir exactamente esta misma regla. El flujo base **siempre** es:

1. Dibujar el fondo estático.
2. Dibujar el *sprite* del jugador moviéndose.
3. Capturar esto en un Canvas limpio.
4. **Dibujar las capas de filtros sobre el Canvas en el orden exacto que se detalla a continuación.**

☀️ Horario: Mañana

1. **Fondo + Jugador** (En el Canvas)
2. manana_1_colorburn.png ➔ Usar Shader personalizado (Modo 2.0)
3. manana_2_add.png ➔ Usar love.graphics.setBlendMode("add")
4. siempre_add.png ➔ Usar love.graphics.setBlendMode("add")

🌇 Horario: Tarde

1. **Fondo + Jugador** (En el Canvas)
2. tarde_1_add.png ➔ Usar love.graphics.setBlendMode("add")
3. siempre_add.png ➔ Usar love.graphics.setBlendMode("add")

🌃 Horario: Noche

Esta es la pila de luz más compleja. Se debe dibujar en este orden estricto:

1. **Fondo + Jugador** (En el Canvas)
2. noche_1_add.png ➔ Usar love.graphics.setBlendMode("add")
3. noche_2_overlay.png ➔ Usar Shader personalizado (Modo 1.0)
4. noche_3_colorburn.png ➔ Usar Shader personalizado (Modo 2.0)
5. noche_4_overlay.png ➔ Usar Shader personalizado (Modo 1.0)
6. noche_5_add.png ➔ Usar love.graphics.setBlendMode("add")
7. siempre_add.png ➔ Usar love.graphics.setBlendMode("add")

(Nota: Asegúrense de devolver el motor a love.graphics.setBlendMode("alpha") después de dibujar las capas "add").

3. Shader Requerido (Overlay y Color Burn)

LÖVE 2D tiene modo "Add" de fábrica, pero **no tiene** *Overlay* ni *Color Burn*. Para que las capas que dicen "Usar Shader personalizado" funcionen y oscurezcan al jugador dinámicamente, deben integrar este código GLSL en su ciclo de renderizado:

```
-- Guardar esto en la inicialización (love.load)
shader_procreate = love.graphics.newShader[[
    extern Image canvas_juego;
    extern vec2 resolucion;
    extern float modo; // 1.0 = Overlay | 2.0 = Color Burn

    vec4 effect( vec4 color, Image textura_filtro, vec2
coords_textura, vec2 coords_pantalla ) {
        vec2 uv = coords_pantalla / resolucion;
        vec4 base = Texel(canvas_juego, uv);
        vec4 blend = Texel(textura_filtro, coords_textura) * color;
        vec3 resultado;

        if (modo == 1.0) { // OVERLAY
            for(int i=0; i<3; i++) {
                if(base[i] < 0.5) {
                    resultado[i] = 2.0 * base[i] * blend[i];
                } else {
                    resultado[i] = 1.0 - 2.0 * (1.0 - base[i]) * (1.0
- blend[i]);
                }
            }
        } else { // COLOR BURN
            for(int i=0; i<3; i++) {
                if(blend[i] == 0.0) {
                    resultado[i] = 0.0;
                } else {
                    resultado[i] = max(1.0 - (1.0 - base[i]) /
blend[i], 0.0);
                }
            }
        }
```

```
        // Mezclar usando la opacidad de la brocha del artista (Canal
Alpha)
        return mix(base, vec4(resultado, base.a), blend.a);
    }
]]
```

Instrucción de uso para el Shader: Antes de dibujar una capa *Overlay*, usen shader_procreate:send("modo", 1.0). Antes de dibujar una capa *Color Burn*, usen shader_procreate:send("modo", 2.0). Siempre pasen el canvas base con shader_procreate:send("canvas_juego", mi_canvas).